Trianz (Deepak V)

Weekly Progress Tracker

Work weeks: Monday to Friday.

Week	Activity & Detail	Status	Notes
Week 1 *08 Jun – 12 Jun*	**HR Decision Support System – POC (8–9 Jun)** Built an end-to-end HR decision support system proof-of-concept using Python, Streamlit, and the Claude API, covering the full stack: data pipeline, rule-based analysis engines, UI, and report generation. Created a functional demo with sample data demonstrating promotion readiness scoring, peer comparison, and compensation analysis. Identified and resolved SDK compatibility and network connectivity issues.	**Done**	POC complete; demonstration ready
	Concierto Lead Intelligence – Kickoff & Scoping (10 Jun) Initiated Concierto Lead Intelligence, an internal AI-powered lead qualification platform for Trianz sales teams. Studied the classic BANT sales framework and extended it to BANT+HS by adding two Trianz-specific dimensions – Scope Clarity (S) and Hyperscaler Alignment (H) – reflecting Trianz's dependency on AWS/Azure/GCP co-sell motions. Defined the tech stack (React + Vite + TypeScript frontend, Node.js + Express + TypeScript backend, Anthropic Claude Haiku 4.5 for AI, PostgreSQL for persistence). Set up the project repository, folder structure, and initial build tooling.	**Done**	Framework and stack defined
	BANT+HS Scoring Engine + Test Suite (11 Jun) Implemented the deterministic client-side scoring engine covering all six dimensions (Budget, Authority, Need, Scope, Timeline, Hyperscaler), each scored 0–3, totalling an 18-point scale mapped to four tiers – HOT (15–18), QUALIFIED (11–14), NURTURE (7–10), DISQUALIFY (≤6). Built hard-rule override logic for edge cases (B=0, A=0, A=1 practitioner-only escalation, N=0+T=0 auto-disqualify) and soft-flag rules (H=0 competitive risk, N=1+T=1 nurture-only, SMB revenue <$500M). Wrote 37 Vitest unit tests covering tier boundaries, hard rules, and signal-parsing edge cases.	**Done**	37 unit tests passing
	Single Lead UI + Claude AI Integration (12 Jun) Built the Single Lead qualification form with real-time scoring, cloud infrastructure tagging (AWS/Azure/GCP/On-Prem/Multi-Cloud), revenue bands, and BANT+HS signal dropdowns. Integrated the Anthropic Claude API using tool-use mode for guaranteed structured JSON output, generating analyst notes, recommended Trianz offerings (Compass/Assess/Migrate/Manage/Modernize), next actions with urgency tags, discovery questions, and outreach email drafts. Added a title-vs-authority sanity check and built the results panel with tier badge, action strip, flag banners, and per-dimension score visualization.	**Done**	AI-generated recommendations live
Week 2 *15 Jun – 19 Jun*	**Bulk Upload Feature + Export Flows (15 Jun)** Built the Bulk Upload module – CSV and Excel parsing with PapaParse and SheetJS, capacity up to 500 leads per batch, and smart column-name detection handling varied header formats. Implemented tier-based filter pills, free-text search, and per-row checkbox selection. Added a cross-lead AI-generated Pipeline Summary and Bulk Email Drafts for selected/top HOT leads. Built PDF (jsPDF), Excel, and CSV export flows for both single and bulk modes.	**Done**	Up to 500 leads per batch
	Cloud Database, Mobile-Responsive UI, Testing (16 Jun) Integrated Azure PostgreSQL for shared team-wide Lead History with a full JSONB schema for scores, flags, and AI analysis. Added localStorage fallback for offline resilience and a one-time email prompt with a 'Sync local → cloud' migration button. Made the application fully mobile-responsive (tested at 375px, 768px, 1280px). Completed end-to-end functional testing, security testing (SQL injection, XSS, header hardening), and grew the test suite to 44 unit tests, all passing.	**Done**	44 unit tests passing

Week	Activity & Detail	Status	Notes
	Azure Infrastructure + Deployment Pipeline (17 Jun) Provisioned Azure resources – App Service (Basic B1, Linux, Node 22, Central India) and Azure Database for PostgreSQL Flexible Server (Burstable B1ms, 32 GiB). Configured firewall rules for App Service–database connectivity. Pushed the project to Trianz GitLab and configured Azure Deployment Center's External Git integration with a scoped read-only PAT. Modified the Express backend to serve the built React frontend from the same origin in production and ran the first deployment.	**Done**	First deployment completed
	Entra ID Authentication + Build Troubleshooting + Hardening (18 Jun) Configured Microsoft Entra ID on the App Service – single-tenant restriction, sign-in redirect for unauthenticated requests, and automatic user-email attribution. Applied HTTP security headers, disabled the X-Powered-By fingerprint, added rate limiting on AI endpoints (500 req/min), and enforced a 1MB request body limit. Debugged Oryx build failures (NODE_ENV=production skipping devDependencies, fixed with --include=dev). Discovered a cross-tenant auth issue (AADSTS50020) and sent a formal request to Trianz IT for App Registration in the main tenant.	**Done**	Cross-tenant issue escalated to IT
	Delivery + Documentation + Next-Project Planning (19 Jun) Resolved remaining deployment blockers – app fully live, authenticated, and stable in production. Coordinated with Trianz IT, who advised B2B guest invites as the sanctioned path for cross-tenant access. Wrote comprehensive project documentation (architecture, features, schema, API reference, security posture, deployment procedure, runbook) and prepared a 5-page test report for stakeholder review. Communicated status to manager and began scoping an upstream Lead Prospecting Agent, producing a detailed project plan.	**Done**	App live at concierto-lead-qualification.azurewebsites.net
Week 3 *22 Jun – 26 Jun*	**Transition to Internal Leadership Dashboard (22 Jun)** Concluded work on the Lead Prospecting/Discovery project and transitioned focus to a new initiative. Onboarded onto the Internal Leadership Dashboard project – a consolidated reporting tool for Trianz leadership covering Finance, Sales, HR, and Delivery metrics. Gathered project context and requirements from Priyesh, including scope, target KPIs, and reference materials (wireframe, sample data). Reviewed the Phase 1 KPI specification (137 metrics across 5 functional areas).	**Done**	137-metric KPI spec reviewed
	Technical Architecture & Data Mapping (23 Jun) Defined the technical architecture: Next.js/React frontend, Python-based data pipeline, JSON data layer, with Claude AI integration for an 'Ask AI' insights panel. Mapped data availability against the KPI specification across Finance, Sales, HR, and Delivery, identifying ~70 ready-to-build metrics and flagging gaps requiring stakeholder input. Initiated stakeholder outreach to Delivery, Finance, and HR teams to source missing data (headcount, backlog, AMM conversion) and set up the project workspace.	**Done**	~70 metrics ready to build
	Session 3 Feature Verification + Bug Fixes (23–24 Jun) Verified all Session 3 visual features end-to-end: QoQ/YoY delta chips, Services/Platforms breakdown footers, drill-down drawers, sparkline charts on 10+ KPI cards, functional filter chips, and the admin refresh log page. Resolved a MarginTable column overflow bug on the Delivery page (removed redundant column, added horizontal scroll, adjusted padding). Conducted a full codebase inventory audit (13 components, 5 tab pages, 6 API routes, 4 ETL parsers) and updated HANDOFF.md with a comprehensive Session 3 closeout.	**Done**	~80 features documented in HANDOFF.md
	HR Stakeholder Review with Ayushi (25 Jun) Collaborated with Ayushi (HR team) for a structured review of People tab metrics, covering headcount, attrition, utilization, and workforce distribution KPIs. Validated data availability in HR source files against the Phase 1 KPI specification, identified gaps between available HR data and required	**Done**	People tab data structure updated

Week	Activity & Detail	Status	Notes
	metrics, and incorporated Ayushi's feedback into the People tab data structure, updating column mappings and metric definitions.		
	Delivery Stakeholder Review with Shobha (26 Jun) Engaged with Shobha (Delivery Lead) to review Delivery tab metrics, covering project health indicators, delivery milestone tracking, and utilization reporting. Evaluated sample Delivery datasets for structure, completeness, and ETL feasibility. Documented field-level mapping between source data and dashboard display requirements, and updated ETL logic and dashboard component structure accordingly.	**Done**	Delivery ETL mapping documented
Week 4 *29 Jun – 03 Jul*	**Resumed Multi-Tab Development (29 Jun)** Resumed development across all functional tabs (Finance, Sales, HR, Delivery) following stakeholder alignment sessions. Worked with Priyesh to review KPI card design, discussing layout, metric grouping, and visual hierarchy for leadership consumption. Clarified open questions on metric definitions and finalized Priority 1 metrics for the initial version.	**Done**	Priority 1 metrics finalized
	UI Build Continuation (30 Jun) Continued UI build across Finance, HR, Delivery, and Sales tabs, focusing on card component reuse, data table structure, and tab navigation consistency. Addressed display formatting issues (currency/percentage formatting, null/empty value handling, label alignment) and worked on visual consistency so the dashboard felt unified across tabs.	**Done**	Unified cross-tab visual style
	Stakeholder Presentation & Feedback (01 Jul) Presented the updated multi-tab dashboard to Priyesh with live data across Finance, HR, Delivery, and Sales. Gathered structured feedback on layout adjustments, additional KPIs to surface, label wording, and priority metric visibility. Documented feedback as action items, began implementing high-priority changes in the same session, and moved key summary cards to the top of each tab for leadership visibility.	**Done**	Feedback logged and partly actioned
	Feedback Implementation (02 Jul) Continued implementing remaining feedback items from the 01 Jul review across tab layouts, chart configurations, and metric card content. Refined data table structures for Delivery and HR tabs (sorting, column widths, readability) and worked on cross-tab formatting consistency – unified font sizes, color usage, card borders, and spacing standards.	**Done**	Delivery/HR tables refined
	ETL Completion & End-to-End Testing (03 Jul) Completed ETL development for all functional tabs – finalized column mappings, data cleaning logic, and output structure for Finance, HR, Delivery, and Sales. Conducted end-to-end testing, running all ETLs against sample data, verifying output JSON matched UI expectations, and resolving parsing edge cases (NaN, blank rows, merged cells). Validated reporting consistency by cross-checking totals against source Excel values and resolved remaining display bugs.	**Done**	All tab ETLs validated end to end
Week 5 *06 Jul – 10 Jul*	**Finance Leadership Presentation (06 Jul)** Presented the current dashboard to Finance leadership, walking through Finance tab KPIs including revenue, billing, and forecast tracking metrics. Finance team raised feedback on metric grouping, additional data cuts (by region, by offering), and clarity of trend indicators. Documented all Finance-specific enhancement requests, began implementing structural changes, and revisited ETL aggregation logic to align with how Finance interprets the numbers.	**Done**	Finance-specific enhancements logged
	Cross-Functional Alignment Meeting (07 Jul) Attended a cross-functional meeting with the CIO (Srikanth), Yaswanth (Business Head), and Prabhat (building the SFDC/Sales portal). Discussed overlap between the Internal Leadership Dashboard and Prabhat's customer-	**Done**	Scope boundary agreed with CIO

Week	Activity & Detail	Status	Notes
	facing Sales portal, identifying shared metrics and duplicate-reporting risks. The CIO directed both teams to align on a single metrics framework; agreed the Internal Dashboard would focus on delivery/leadership KPIs and the Sales portal on SFDC pipeline/customer-facing metrics.		
	Metrics Comparison Analysis (08 Jul) Performed a detailed side-by-side comparison of metrics between Internal Dashboard tabs and Prabhat's Sales portal, reviewing field names, definitions, data sources, and calculation logic. Identified 12+ overlapping metrics with different definitions or sources, compiled a structured Excel comparison document (metric name, source, owner, definition, recommended action), and submitted it to Priyesh for review and final ownership decisions.	**Done**	12+ overlapping metrics flagged
	Consolidated Framework Implementation (09 Jul) Received the consolidated metrics framework from Priyesh with final decisions on metric ownership and definitions. Separated Excel source sheets by team to prevent future confusion, updated ETL column mappings and calculation logic to align with the consolidated definitions, and created a clean field-level ownership reference document shared with Prabhat for portal alignment.	**Done**	ETL mappings aligned to framework
	Alignment Finalization + Platform Sales Kickoff (10 Jul) Collaborated with Prabhat to walk through the consolidated metrics framework and confirm alignment on shared fields, then prepared and submitted the final Excel comparison for sign-off. Implemented a month-level filter on the Finance tab (previously only year/quarter), updating ETL aggregation for monthly slicing. Initiated the Platform Sales initiative – began designing the Backlog and Active Engagements tab architecture and identifying required data sources (Forecast Tracking FY27, AWS Master Billing).	**Done**	Platform Sales initiative kicked off
Week 6 *13 Jul – 17 Jul*	**Scope Finalization – Backlog & Active Engagements (13 Jul)** Attended a follow-up planning meeting with Yaswanth and Prabhat, finalizing the scope boundary: Backlog and Active Engagements tabs to be built by Deepak and integrated into Prabhat's production portal. Confirmed data sources (Forecast Tracking FY27 for Backlog; AWS Master Billing + AWS Assess Migrate for Active Engagements), mapped Priyesh's Platform delivery KPI specs to source columns, and began the tab framework build (route structure, ETL stubs, component scaffolding).	**Done**	Scope and data sources confirmed
	ETL Build – Backlog & Active Engagements (14 Jul) Built the Backlog tab ETL (etl/platforms_backlog.py) with sheet-level parsing of Forecast Tracking FY27, dedup logic by Project Code, and ACV aggregation by channel and offering. Built the Active Engagements ETL (etl/active_engagements.py), parsing the Append1 sheet from AWS Master Billing with an SFDC join for VM scope data and a Forecast Tracking fallback join. Engaged IT (Saravanavignesh) to initiate SharePoint app registration on the test tenant and began building the SharePoint client (Microsoft Graph API credentials flow, site discovery, in-memory file download).	**Done**	Both ETLs built; SharePoint client in progress
	UI Completion + SharePoint Testing (15 Jul) Completed Backlog and Active Engagements tab UI components – summary cards, KPI tiles, sortable data tables, and chart components. Completed SharePoint connectivity testing (10/10 test cases passed: authentication, site access, file discovery, in-memory download, ETL end-to-end, auto-startup, local fallback). Uploaded all source files to the test SharePoint site, generated a test results PDF for IT, and pushed the full project to GitLab, sharing the repository with Prabhat for portal integration.	**Done**	10/10 SharePoint test cases passed
	Production Integration + VA Testing (16 Jul)	**Done**	VA testing passed, no critical findings

Week	Activity & Detail	Status	Notes
	Prabhat integrated the Backlog and Active Engagements tabs into his production portal; collaborated over Teams to resolve integration questions (env vars, Python dependencies, ETL run commands). The application was submitted for vulnerability assessment (VA) testing and passed with no critical findings. Conducted post-integration validation, confirming production data matched local ETL output and that SharePoint reads worked in the hosted environment, and continued refining data accuracy.		
	Deal Portfolio Requirement + New ETL Build (17 Jul) Presented integration completion and VA results to Priyesh. Received a new high-priority requirement: replace the Active Engagements list (from AWS Master Billing) with the Deal Portfolio Excel as the primary data source, since it contains richer delivery context (blockers, remarks, migration scope). Assessed the Deal Portfolio structure (Direct/Partner sheets) and began building etl/deal_portfolio.py with dual-sheet parsing and Claude Haiku–based recommended next steps per deal, and started redesigning the Active Engagements tab.	**Done**	Deal Portfolio ETL build started
Week 7 *20 Jul – 24 Jul*	**Deal Portfolio ETL + UI Components (20 Jul)** Completed etl/deal_portfolio.py – finalized column parsing for Direct and Partner sheets and implemented per-deal AI recommendations via Claude Haiku, output to data/deal_portfolio.json. Built DealPortfolioTable.tsx (expandable remarks, deal detail modal, phase chart modal) and a DealPortfolioPreview component. Wired the AI Leadership Summary card to both Backlog and Active Engagements tabs. Pushed updates to GitLab for Prabhat to deploy, and discussed Finance automation opportunities with Shreyansh for a future phase.	**Done**	Deal Portfolio live in production
	UAT Feedback Implementation (21 Jul) Addressed UAT feedback from the stakeholder review of the production URL: validated empty/null value display on the Backlog tab, added a data-refresh disclaimer, made table headers sortable, and added color-coded completion percentages on the Active Engagements tab. Simplified sourcing text across components, fixed the Phase Chart modal (single chart per Direct/Partner tab with independent scope/completed metrics per phase), and wired the AI panel to load the correct JSON per tab. Pushed all fixes to GitLab for deployment.	**Done**	UAT feedback resolved and deployed
	Needs Attention Panel + Action-Oriented Features (22 Jul) Aligned with Priyesh on evolving the dashboard from informational to action-oriented. Implemented NeedsAttentionPanel.tsx, flagging up to 5 deals with active blockers, low assessment/migration completion, or deals ready to move phase. Enhanced the AI Leadership Summary with structured heading/sub-point formatting and added a deal categorization action table (Drive Migration Closure, Get Clarity on Rationale & Timeline, Customer Connect Missing) with sales owner shown inline. Pushed all changes to GitLab for deployment.	**Done**	Action-oriented panel deployed
	Portal Review with Sheetal (23 Jul) Conducted a structured portal review session with Sheetal (Delivery Lead) and Priyesh, demonstrating the Active Engagements tab, Deal Portfolio table, AI Leadership Summary, and Needs Attention panel. Sheetal raised feedback on migration rationale/urgency visibility, customer engagement gap identification, and surfacing of project timelines/deadlines. Priyesh prioritized all three items for the next sprint, and detailed requirements (urgency classification logic, field mappings, customer-connect derivation) were documented.	**Done**	Next sprint requirements documented
	Migration Urgency Classification + Performance Optimization (24 Jul) Implemented a 4-level Migration Urgency & Timeline badge (Red/Amber/Yellow/Green) based on reason_for_migration and customer_deadlines fields, expandable to show full driver/deadline text.	**Done**	ETL runtime reduced from ~3 min to ~15 sec

Week	Activity & Detail	Status	Notes
	Restructured the Deal Details modal to separate AI action bullets from customer context. Updated the AI prompt to add decision_maker extraction and always pass deadline/driver context. Parallelized ETL AI calls with ThreadPoolExecutor (58 deals in ~15 seconds, down from ~3 minutes), added a 'no customer connect' flag to the Needs Attention panel, and fixed the '+N more' expand control. Pushed as commit 3538746; Prabhat completed final integration and redeployed.		
Week 8 *27 Jul – 31 Jul*	**Consolidated Stakeholder Review (27 Jul)** Conducted a comprehensive review of the Internal Leadership Dashboard with Sheetal, Priyesh, and Pradeep Bhat, walking through all active tabs — Active Engagements, Backlog, Deal Portfolio, and the AI Leadership Summary panel. Gathered consolidated feedback on data accuracy, visual presentation, and metric prioritization. Documented and prioritized all feedback items and began implementing high-priority changes, including label refinements, card layout adjustments, and metric visibility improvements across tabs.	**Done**	Feedback documented and implementation started
	Executive Summary Refinements (28 Jul) Continued implementing feedback from the 27 Jul review. Refined the Executive Summary tab — updated KPI card groupings, improved trend indicator visibility, and ensured cross-tab formatting consistency. Validated updated components against live data to confirm metric accuracy, and addressed ETL edge cases (null handling, formatting inconsistencies) surfaced during the review. Coordinated with Priyesh on prioritization of remaining items and confirmed scope for the next sprint.	**Done**	Cross-tab consistency improvements applied
	Dashboard Refinements & Internal Testing (29 Jul) Continued dashboard refinements following stakeholder feedback, improving data readability across Finance, HR, and Delivery tabs — adjusted chart configurations, refined table column widths, and standardized label formats. Reviewed ETL outputs for all tabs to confirm data integrity after recent changes. Conducted internal testing of the full dashboard flow (tab navigation, filter interactions, AI summary generation, PDF export) and resolved minor display bugs in the HR attrition section and the Delivery utilization card.	**Done**	Internal testing completed; display bugs resolved
	AWS Billing Dashboard Support – Deployment & Auth (30 Jul) Supported Bhuvaneshwari with her AWS Billing Dashboard project under Finance. Assisted with Azure App Service deployment configuration, resolving build errors related to dependency installation and environment variable setup. Helped configure Microsoft Entra ID authentication with correctly scoped Trianz tenant access. Reviewed and tested key features (fund request pipeline tracking, attention-required flags, billing status cards). Continued parallel work on the Internal Leadership Dashboard, applying remaining feedback items and pushing updates to GitLab.	**Done**	AWS Billing Dashboard deployment/auth resolved; Internal Dashboard updates pushed
	End-to-End Deployment Testing (31 Jul) Initiated end-to-end deployment testing of the Internal Leadership Dashboard on Azure App Service — tested the full pipeline (GitLab push to Azure sync, Oryx build process, startup command execution, health check). Identified and began investigating startup issues related to Python package availability on the Azure Node.js runtime image. Continued assisting Bhuvaneshwari on the AWS Billing Dashboard, reviewing additional feature requirements and troubleshooting data refresh/display issues. Documented deployment findings and outstanding blockers for follow-up.	**In Progress**	Deployment blockers identified; investigation ongoing
Week 9 *03 Aug – 07 Aug*	**Azure Python/Pip Investigation (03 Aug)** Resumed deployment work on the Internal Leadership Dashboard. Investigated the root cause of Python package installation failures on Azure's Node.js runtime image — identified that the image ships with Python 3.11 but without pip, requiring a custom installation approach via startup script.	**In Progress**	Python/pip installation on Azure Node.js image — active investigation

Week	Activity & Detail	Status	Notes
	Iterated on multiple startup script configurations, testing pip bootstrapping methods. In parallel, continued work on the AWS Billing Dashboard, reviewing feature completeness and supporting Bhuvaneshwari with remaining deployment and configuration tasks.		
	SharePoint ETL Extension to All Tabs (04 Aug) Continued resolving the Azure deployment pipeline for the Internal Leadership Dashboard. Worked on enabling the Python ETL refresh pipeline on Azure — the ETL layer pulls live data from SharePoint via Microsoft Graph API and writes JSON for the dashboard to consume. Extended SharePoint integration to cover Finance, HR, Delivery, and Sales ETLs (previously only Active Engagements and Backlog had SharePoint connectivity). Tested ETL connectivity locally — all four new ETLs confirmed pulling from SharePoint successfully. Continued supporting the AWS Billing Dashboard alongside this work.	**In Progress**	SharePoint integration extended to all tabs; Azure ETL pipeline deployment in progress
	Pip Bootstrapping + New Project Scoping (05 Aug) Continued resolving the Azure deployment pipeline, investigating pip bootstrapping on the Azure Node.js runtime image — Python 3.11 is present but pip is completely stripped, requiring installation via get-pip.py at container startup. Iterated on startup script configurations to reliably install pandas, openpyxl, anthropic, and requests on the Azure container, working toward a fully operational ETL refresh pipeline pulling live SharePoint data across all tabs. Continued supporting the AWS Billing Dashboard alongside deployment work. Had an initial connect with Siddhartha Pendyala regarding a new upcoming SFDC intelligence project covering sales stage insights and delivery/finance analytics — early scoping stage.	**In Progress**	Azure pip installation actively being resolved; ETL refresh pipeline deployment ongoing
	AI Leadership Summary — All Tabs (06 Aug) Extended the AI Leadership Summary card to Finance, HR, Delivery, and Executive tabs (previously only Active Engagements and Backlog had this feature). Each tab now auto-generates a structured 3-section AI summary on page load with tab-specific prompts (revenue/margin for Finance, attrition risk/headcount gaps for HR, utilization/bench concerns for Delivery, cross-functional aggregate for Executive). Updated LeadershipSummaryCard to accept a custom question prop for per-tab prompt customization. Separately resolved a 504 Gateway Timeout on Azure caused by the startup script installing python3-pandas via apt-get (pulling 100+ unrelated dependencies and blocking Next.js startup) by moving package installation to a background subprocess so Next.js starts immediately while packages install asynchronously into /home/pypackages. Pushed and deployed to Azure.	**Done**	AI Leadership Summary live on all tabs; 504 Gateway Timeout resolved
	Azure Deployment Investigation (07 Aug) Continued investigating "Data pending" across all dashboard tabs on Azure. Traced the root cause to two compounding issues — the data/ folder is gitignored so no JSON files exist on Azure, and the ETL refresh pipeline fails because Python packages (pandas, openpyxl, anthropic) are unavailable on the Node.js runtime image. Reviewed multiple approaches and evaluated long-term architectural options, scoping a production-grade solution: migrating ETL execution to Azure Functions (Python 3.11 runtime with pip natively available), storing ETL output in Azure Blob Storage, and updating the dashboard to read from Blob with a local file fallback for development — eliminating the Python-on-Node problem permanently.	**In Progress**	Architecture finalized; Azure resource setup and implementation planned for next week
Week 10 *10 Aug – 14 Aug*	**Internal Dashboard Deployment & Architecture (10 Aug)** Continued resolving the Azure deployment pipeline. Investigated the persistent "Data pending" issue across all tabs post-deployment, tracing it to Python packages being unavailable on the Azure Node.js runtime, preventing the ETL refresh pipeline from executing. Iterated on startup script configurations and evaluated approaches to reliably install pandas, openpyxl, anthropic, and requests on the Azure container. Finalized and implemented a background installation approach — packages install	**In Progress**	Background pip install deployed; end-to-end ETL refresh testing ongoing

Week	Activity & Detail	Status	Notes
	asynchronously into /home/pypackages (persistent across restarts) while Next.js starts immediately, eliminating the 504 Gateway Timeout. Pushed fixes to GitLab, redeployed, and completed internal testing of the full dashboard flow post-deployment.		
	Dashboard Changes & AWS Billing Dashboard Completion (11 Aug) Continued refinements to the Internal Leadership Dashboard — applied cross-tab consistency improvements, validated ETL outputs for Finance, HR, Delivery, and Sales tabs, and confirmed AI Leadership Summary cards generating correctly on all tabs post-deployment. In parallel, completed final work on the AWS Billing Dashboard project with Bhuvaneshwari — resolved remaining deployment and authentication issues on Azure App Service, confirmed Microsoft Entra ID SSO working correctly with Trianz tenant scoping, and validated all key features (fund request pipeline tracking, attention-required flags, billing status cards, data refresh). The AWS Billing Dashboard was signed off and handed over as complete.	**Done**	AWS Billing Dashboard projec completed and signed off
	Azure Functions & Blob Storage Architecture Implementation (12 Aug) Began implementing the long-term production-grade ETL solution — migrating from Python subprocess execution on the Node.js server to Azure Functions (Python 3.11 runtime) with Azure Blob Storage for JSON persistence. Scoped Azure resource requirements, created a Storage Account and Blob container (dashboard-data), and began building the Function App with timer-triggered ETL functions for Finance, HR, Delivery, Sales, Active Engagements, and Backlog. Updated dataCache.ts to read from Blob Storage with a local file fallback for development, keeping the local dev workflow unchanged. Completed and prepared the Internal Leadership Dashboard documentation (15-section Word document covering architecture, ETL pipeline, SharePoint integration, data dictionary, user guide, and troubleshooting).	**In Progress**	Azure Functions build underway; Blob Storage integration in progress
	FR Portal — Project Kickoff & Architecture Study (13 Aug) Initiated the AWS ISV Fund Request Portal project — a centralized web application replacing the current Microsoft Lists-based workflow for tracking AWS ISV Tooling Fund Requests. Studied the architecture and product design document (v1.2), covering the three-role system (Seller, Funding Ops, Leadership), the high-level architecture (React + TypeScript frontend, Node.js/NestJS backend, PostgreSQL, Azure Functions for integrations, SharePoint/Blob Storage for file handling), and the phased delivery roadmap. Reviewed the problem statement and MVP success criteria, mapped user personas to portal access levels, and began setting up the project repository, folder structure, and initial tooling. Completed Concierto platform training and cloud migration learning modules.	**In Progress**	FR Portal kickoff complete; architecture reviewed and repository set up
	FR Portal — Deep Dive & Initial Build Setup (14 Aug) Continued FR Portal scoping and build preparation — reviewed the full module breakdown (seller submission form, Funding Ops Excel sync, leadership dashboard, email notifications, admin panel), studied the data model (core entities: FundRequest, User, StatusHistory, AuditLog), and examined the integration architecture (SharePoint Excel watcher via Azure Functions, email dispatch via Azure Communication Services, future AI layer). Began designing the database schema and API structure aligned with the architecture document. Completed additional cloud migration and AWS ISV programme training to build context for the fund request domain.	**In Progress**	FR Portal module design started; schema and API design in progress

Week 11 *17 Aug – 21 Aug*	**FR Portal — Codebase Assessment & Backend Scaffold (17 Aug)** Scanned the existing FR Portal frontend repo (React + Vite, seller submission form already scaffolded) and assessed it against the full architecture scope — identified that backend, database, auth, and email were entirely unbuilt. Reconciled the schema design with explicit requirements	**Done**	Backend scaffold and schema design complete; build sequence agreed; Internal Dashboard ETL refresh monitored on Azure

	(three separate aws_stage/aws_status/aws_fund_stat fields rather than a single merged status, composite unique key on sfdc_id + fund_id for the Excel sync's matching logic). Scaffolded the NestJS backend project structure and Prisma schema, and planned the build sequence: schema → API → email triggers → dashboard. Continued monitoring the Internal Leadership Dashboard deployment on Azure — verified background Python package installation completing successfully in logs and conducted initial ETL refresh testing post-deployment.		
	Prisma Schema, Auth & Fund-Request API (18 Aug) Built the full Prisma schema (fund_requests, sync_log, notifications, users) and the NestJS backend: mock-login issuing a JWT shaped identically to a future Azure AD/MSAL token, role guards, and the core fund-request endpoints (create/list/detail/status-update). Wired the ACS email SDK for the three required triggers (submission confirmation, FundingOps alert, status-change), wrapped in try/catch so a failed send never breaks a request. Verified the full flow end-to-end using an in-memory mocked Prisma client since a real Postgres/Prisma client generation wasn't yet available locally. Made minor Internal Leadership Dashboard refinements — confirmed AI Leadership Summary generating correctly across Finance, HR, Delivery, and Executive tabs on Azure and validated cross-tab data consistency. Began cloud computing training — completed Module 1 covering core cloud computing concepts (delivery of servers, storage, networking, databases, software over the internet on a pay-as-you-go basis) and the difference between public, private, and hybrid cloud models.	**Done**	Backend + API + email triggers verified via automated mock-DB test suite; cloud computing training started
	Schema Correction, Real Postgres, and Live Data Verification (19 Aug) Corrected the Engagement Type field from a joined string to a proper Postgres array after confirming the seller form's UI is genuinely multi-select, and re-verified the fix. To get past a corporate proxy blocking Prisma's native binary downloads, stood up a portable local Postgres instance, hand-applied the schema via SQL, and imported real production data from AWS Master Billing.xlsx to validate the schema against actual data — this surfaced a real data-quality issue (repeated sfdc_id+fund_id pairs in the source sheet) now flagged for the Excel-sync design. Manually filled and submitted the real seller form in a browser, confirming data landed correctly end-to-end. Continued cloud computing training — completed Module 2 covering core cloud service categories (compute, storage, networking, managed databases) and their use cases.	**Done**	Schema validated against real production data; one data-quality decision flagged for sync design; 2 of 7 cloud computing modules completed
	Dashboard, Admin Module, Production Hardening & Legal Automation Kickoff (20 Aug) Built the FundingOps/Leadership dashboard (Kanban view by stage, KPI cards, filters) and the Admin module (user role/deactivation management, audit-log viewer), backed by two new API endpoints and covered by automated tests. Added production-hardening items deferred from the initial build: per-user rate limiting, structured JSON logging with request-id correlation, a DB-backed health-check endpoint, a CORS allowlist plus helmet, and a CI workflow to run the test suite automatically on push. Found and fixed four real bugs during testing: an auth session dropping on page refresh, a raw 500 on duplicate submissions, an 'Expiring Soon' panel showing already-expired items, and a native HTML radio/label pattern causing a full-page navigation on the form's Yes/No toggles (root-caused across three browsers, fixed by switching to plain buttons). Attended an initial requirement-gathering meeting for a new project — Legal Department Automation — covering current legal workflows at Trianz, key pain points around contract review, document management, and compliance tracking, and early thoughts on automation scope. Documented initial requirements and open questions for follow-up scoping.	**Done**	Dashboard, admin module, and hardening complete; 47 automated checks passing; Legal Automation requirement gathering initiated

	Repository Setup, Handoff Documentation & Cloud Training (21 Aug) Pushed the completed FR Portal project to the GitLab repository (fundportal/fundportal) for Bhuvaneshwari's integration. Authored a detailed handoff document covering the full build inventory, exact run/deploy commands and environment variables, and two decisions flagged for her sign-off: the duplicate-key handling approach for the Excel sync, and a redundant status field removed from the seller form to avoid conflicting with sync-written data. Documented the one open item clearly — the real NestJS+Prisma backend process has not yet booted end-to-end due to the local network's Prisma binary block, verified instead through automated mocked-DB tests and one real-database manual pass through a lightweight test harness. Continued cloud computing training — reviewed and consolidated learnings from the two completed modules on scalability, cost efficiency, agility, security, and business continuity benefits of the pay-as-you-go model. Remaining 5 modules planned for continuation in Week 12.	**Done**	Repository and handoff documentation delivered; awaiting Bhuvaneshwari's environment verification and sign-off on two flagged decisions; cloud training ongoing (2/7 modules complete)

Week	Activity & Detail	Status	Notes
Week 12 *24 Aug – 28 Aug*	**Cloud Computing Course (24–25 Aug)** Completed a 4-day cloud computing course covering core cloud infrastructure concepts, deployment models, virtualization, and cloud service fundamentals. The Legal Department Automation project was formally marked on hold and not being continued further for now.	**Done**	Cloud training completed; Leg Automation project put on ho
	FR Portal Resumption & Handoff Review (26 Aug) Resumed work on the FR Portal after Bhuvaneshwari had made significant updates to the project in a separate, newer repository. Went through her handoff documentation and a full email thread with IT summarizing everything requested and provisioned so far — the Azure AD App Registration for login (Client ID, Tenant ID), the local development redirect URL already configured, and a note that the project would not use multi-tenant authentication. Also reviewed a SharePoint test site IT had provisioned for validating the Excel-sync integration once that work resumed.	**Done**	Handoff docs and IT provisio reviewed
	Full Codebase Audit & Comparison (27 Aug) Did a detailed file-by-file comparison between the originally built version and Bhuvaneshwari's updated version. Found genuine improvements — the auth guard was rewritten to re-check the user's active status against the database on every request rather than trusting the token's embedded role, closing a gap where a deactivated user could keep using a valid session. On the schema side, an unused 'leadership' role was removed and a new 'rejected' stage was added. Flagged two cleanup items — an unnecessarily complex database connection pattern, and a plaintext password left inside an infrastructure provisioning script.	**Done**	Improvements verified; 2 clea flagged
	Deployment Blockers Identified & First Local Run (28 Aug) Found a real bug in the backend's TypeScript build configuration — missing settings for the compiler's source root, causing compiled files to land one folder deeper than expected, which would have made the Azure deployment fail outright. Separately, discovered a required database driver dependency had become corrupted during an earlier fix attempt, traced to the corporate network's security proxy blocking the download. Despite both issues, got the real backend application running end-to-end on a local test setup for the first time since the project changed hands.	**In Progress**	2 deployment blockers found; confirmed working locally

Week 13 *31 Aug – 4 Sep*	**Backend Deployment Attempt & Manual Fix (31 Aug)** Began deploying the FR Portal backend to Azure App Service, configuring all environment variables (database connection string, JWT signing secret, Azure AD tenant/client IDs, allowed CORS origins). Azure Deployment Center's automatic Git deployment repeatedly failed — the build system kept detecting and building the frontend instead of the backend due to the monorepo layout, tried three times with different configs. Abandoned automatic deployment and switched to manual: cloned the repo in Cloud Shell, built the backend, zipped the compiled output, and deployed it directly. Backend ran for the first time on real Azure infrastructure, but the health-check returned a 'database unreachable' error.	**In Progress**	Manual deployment succeede connectivity issue found
	Backend Fully Live & Database Setup (1 Sep) Diagnosed the 'database unreachable' error — firewall rules were already correct, but the configured database name didn't match the real database name on the server. Corrected it, restarted the backend, and confirmed via health-check that it was fully live and connected to the real Azure PostgreSQL database. Ran the schema setup command to create every required table (users, fund requests, sync logs, notifications) against the live, empty database, and manually inserted one admin user so an account existed to test login. Wrote up the full deployment process as a runbook, plus a reusable build-and-deploy script.	**Done**	Backend fully live and connec Azure PostgreSQL
	Frontend Deployment (2 Sep) Weighed Azure App Service versus Azure Static Web Apps for hosting the React frontend, and concluded Static Web Apps was the better fit — purpose-built for SPAs, cheaper, comes with a built-in CDN, and needs no continuously running server. Added a routing configuration file so the app would behave correctly on browser refresh of an internal page instead of showing a 'not found' error. Built the frontend with production settings and deployed it to Static Web Apps successfully. Both frontend and backend were now live and reachable on the public internet for the first time since the project changed hands.	**Done**	Frontend live on Static Web A stack publicly reachable
	Authentication Blocked: Login Flow Investigation (3–4 Sep) Tested logging in with a real Microsoft company account. 'Sign in with Microsoft' redirected correctly and credentials were accepted, but instead of landing back inside the app it silently looped back to the login screen with no error. Used browser DevTools to trace network requests and found the crucial request exchanging the Microsoft login result for app access was never being sent. Verified the backend's own login endpoint was working correctly in isolation, ruling out the backend. Narrowed the cause to two stacked issues, the first being that the Azure AD App Registration had been set up as a standard 'Web' application instead of a 'Single-page application,' silently blocking the required OAuth flow.	**In Progress**	SSO blocker partly root-cause Registration type issue found
Week 14 *7 Sep – 11 Sep*	**Deployment Setup + First Attempts at the SSO Blocker (7 Sep)** Attempted to serve the frontend from App Service, then reverted after confirming Static Web Apps was the right target. Added staticwebapp.config.json for SPA routing and wrote DEPLOYMENT.md, the deployment runbook, from scratch, plus a redeploy-backend.sh script (build, verify dist/main.js, zip, deploy). First SSO debugging attempt — switched MSAL cache to localStorage and enabled an auth-state cookie fallback — did not fix it. Added temporary debug logging to trace MSAL redirect handling	**In Progress**	Deployment path working; SS unresolved

	and fixed a TypeScript build failure from an invalid MSAL config property. Deployment path now working, but login remained broken after a day largely consumed by the SSO bug.		
	SSO Fixed, First Defect Round, ACS Hardened (8 Sep) Root-caused the SSO blocker — no route existed for '/', so Microsoft returned the auth response in the URL fragment, React Router fell through to the wildcard and redirected to /login before MSAL ever read it. Fixed by withholding the router until handleRedirectPromise() resolves. Found a second, independent cause — the deployed bundle was a React development build because Cloud Shell presets NODE_ENV=development. Fixed an HTTP 500 on date fields, replaced a fake auto-save indicator with real localStorage draft persistence, made the Rejected stage reachable, fixed the test suite (which couldn't run at all), and made the ACS email path degrade to mock instead of crashing on bad config.	**Done**	Login working in production, across all three roles
	Review Workflow + Defect Sweep + Excel Sync Begins (9 Sep) Built a role-aware field-edit endpoint, centralised field ownership in one table, and made the New Request form reusable for both create and edit screens. Built the request detail page and wired notification emails to deep-link to it; audit trail now records the actual actor. Drove the real UI and found 17 defects — a 'read-only' screen still editable, logout leaking one user's draft to the next, an admin able to lock themselves out, capital-T text truncation, an unreachable Save button, and new requests missing from the list after submit. Enforced the 11 required fields properly, removed Stage/Status/Fund Stat editing since they come from Excel, and made submitted requests final for sellers. Started the Excel sync — fixed it wiping aws_fund_stat on 213 rows and six further defects.	**Done**	17 UI defects fixed; Excel syn underway
	Excel Sync Completed and Live in Production (10 Sep) Fixed two Function App deployment blockers and a blank Amount being written as a value, erasing funding figures. Built the Excel Sync upload page for Funding Ops and Super Admin, server-enforced, and extracted one shared sync engine used by both the manual upload and the scheduled Azure Function, with a build-time drift check. Verified the upload over real HTTP, fixed a false 'needs manual linking' report, and built an import that creates a request for every fund nobody had raised. Created the Function App in Azure and corrected the admin-seed script. Result: 119 funds live on the dashboard, reconciling to the sheet to the cent, with 0 errors and 306 automated checks passing.	**Done**	119 funds live; $8,803,184 rec the cent; 306 checks passing
	Deployment and Production Verification (11 Sep) Pushed the Excel sync work and redeployed both backend and frontend. Ran the first production upload of the real AWS Master Billing workbook and confirmed 119 requests were created, then re-uploaded the same file to confirm idempotency — nothing written, no duplicates. Reconciled dashboard KPIs and every stage bucket against the workbook, and verified role separation in production (Excel Sync hidden from sellers and refused at the API). Confirmed imported requests render correctly on the dashboard and detail pages, circulated updated handoff documentation, and planned the ACS email work as the next item.	**Done**	Excel sync verified idempoten production; ACS email planne
Week 15 *14 Sep – 18 Sep*	**ACS Provisioning Begins (15 Sep)** Reviewed the existing email implementation and its three triggers (new-submission alert, submission confirmation, status change). Created the Azure Communication Services resource fr-portal-acs	**Done**	ACS and Email Communicati resources created

	and the Email Communication Service fr-portal-email, both in the India region. Reviewed managed-domain versus custom-domain options and chose the managed subdomain for testing, and reviewed the required app settings (ACS_CONNECTION_STRING, ACS_SENDER_ADDRESS, ACS_MOCK).		
	Domain Provisioning and Verification (16 Sep) Provisioned the Azure managed subdomain and completed domain verification — Domain, SPF, DKIM, and DKIM2 all showing Verified. Connected the domain to the fr-portal-acs resource, configured the DoNotReply sender username, and retrieved the ACS connection string and endpoint. Submitted a test fund request to confirm the email pipeline fires end to end — recorded as mocked, confirming the code path was reached while still in mock mode. Prepared the App Service settings for the switch to live sending.	**Done**	Domain fully verified; ready f sending switch
	ACS Email Brought Live (17 Sep) Compared the re-downloaded repo against the pushed codebase — byte-identical, confirming ACS work was configuration only. Forms submitted successfully but no mail arrived; verified every app setting and confirmed the domain's SPF/DKIM were all still verified. Enabled application logging and found rows marked 'sent' with real Azure message IDs, proving the mock-to-live switch had taken effect, which ruled out mail-filter causes. Found and fixed the real defect (commit a5cc41d) — EmailService.deliver() returned the ACS message ID without checking result.status, so rejected mail was being recorded as sent; fixed to throw unless status is Succeeded. Redeployed and confirmed a test submission delivered successfully with correct content and a working deep link. Expanded the recipient list to three addresses.	**Done**	Email delivery confirmed wor recipient list expanded to 3
	GTM Lead Intelligence — Architecture & Solution Planning (18 Sep) Reviewed the GTM Lead Intelligence Agent requirements document — user workflows, lead discovery, scoring logic, delivery channels, and AI requirements. Evaluated Microsoft Power Platform (Copilot Studio + Dataverse) against a custom application architecture, and finalized the approach using Next.js, Azure PostgreSQL, Azure OpenAI, Power Automate, and RSS-based public news sources. Designed the end-to-end workflow — preferences, signal mapping, RSS discovery, deduplication, AI analysis, industry validation, lead scoring, dashboard consumption, and email delivery — and defined the application modules, database entities, API structure, and dashboard pages.	**Done**	Solution architecture finalized Azure PostgreSQL + Azure O selected
Week 16 *21 Sep – 25 Sep*	**GTM Lead Intelligence — Frontend Prototype Development (21 Sep)** Built and refined the GTM Lead Intelligence frontend prototype, implementing the Dashboard, Lead Explorer, Lead Details, Preferences, Search History, and Settings pages. Added interactive lead search, filtering, column customization, signal mapping management, and Run Now workflow states. Updated branding from GTMIQ to Trianz GTM Lead Intelligence and aligned UI terminology throughout. Refined the sample lead dataset and made dashboard metrics dynamically driven by available data, added profile management with saved values reflected across the app, and improved responsive behavior and mobile navigation. Validated interactions and completed a successful production build.	**Done**	Frontend prototype completed PostgreSQL and Azure OpenA integration

Week 16	**GTM Lead Intelligence — Backend Infrastructure & Database Setup (22 Sep)** Set up the complete backend infrastructure for the Trianz GTM Lead Intelligence application. Provisioned Azure PostgreSQL Flexible Server and designed the full database schema covering users, preferences, articles, leads, search history, and offering_signals tables. Implemented the migration runner system with checksum validation to ensure safe, sequential, idempotent schema migrations. Built all core repository layers — user, preference, article, lead, signal, and search history repositories — with full TypeScript typing. Configured the Azure App Service (Central India) for Node.js deployment and wired environment variables for database connectivity, Azure OpenAI, and workspace identity.	**Done**	DB schema finalized; Az provisioned
	GTM Lead Intelligence — API Layer, Discovery Engine & Google News Integration (23 Sep) Built the complete REST API layer using Next.js App Router — endpoints for workspace, preferences, signals, leads, dashboard, search history, and discovery trigger. Implemented the Google News RSS discovery source with full XML parsing, HTML entity decoding, and article normalization. Built the deduplication service using content hashing and canonical URL normalization to prevent duplicate articles across runs. Implemented the query generation service — converting user preferences and DB signals into targeted Google News search queries. Integrated Azure OpenAI GPT-4o for article analysis — built the system prompt, user prompt, and JSON response parsing for lead scoring (Very Hot / Hot / Warm) with industry and region validation.	**Done**	Full discovery pipeline w end-to-end locally
	GTM Lead Intelligence — Full Stack Deployment to Azure App Service — v1.0 (24 Sep) Completed the full application build and deployed Trianz GTM Lead Intelligence v1.0 to Azure App Service (Central India). Integrated the scheduler service via Next.js instrumentation — automatically triggers lead discovery at user-configured IST schedule times. Verified the production build passes TypeScript checks and Next.js compilation. Confirmed database connectivity from Azure App Service to Azure PostgreSQL Flexible Server with SSL. Validated the complete end-to-end flow: preferences saved → signals loaded → Google News RSS fetched → GPT-4o analysis → leads stored → dashboard updated.	**Done**	v1.0 live on Azure — ini pushed to GitLab
	GTM Lead Intelligence — Bug Fixes: Deduplication, Timezone, Email Digest & Azure Build (25 Sep) Resolved four production issues identified after v1.0 deployment. Lead deduplication — fixed duplicate leads appearing for the same company and offering across multiple discovery runs; implemented company+offering key deduplication per user per run. Timezone handling — schedule times now entered in IST and automatically converted to UTC for storage, displayed back as IST in the UI. Azure Communication Services email digest — implemented post-discovery email summary using ACS, sending a formatted lead digest to the user's configured notification email after each run; fixed the EmailService bug where rejected emails were being logged as sent due to unchecked result.status. Azure build pipeline — fixed dev dependencies not installing during Azure App Service build, causing runtime failures on production.	**Done**	All four fixes deployed a on production

Week 17 *28 Sep – 2 Oct*	**GTM Lead Intelligence — Azure AD App Registration & SSO Architecture Setup (28 Sep)** Initiated enterprise authentication implementation for the GTM Lead Intelligence application. Set up the Azure Active Directory App Registration — configured client ID, client secret, tenant ID, and OAuth 2.0 redirect URIs for both local development and the production Azure App Service URL. Defined the required permission scopes (openid, profile, email). Researched and selected NextAuth.js as the authentication framework for Next.js App Router. Designed the multi-user data architecture — mapping authenticated Azure AD users to internal DB records by email, and scoping all leads, preferences, and search history to individual user IDs to ensure complete data isolation between team members. Drafted the full auth flow: Microsoft login → Azure AD token → NextAuth JWT callback → user upsert in PostgreSQL → session with userId.	**Done**	Azure AD app registratio complete; auth architectu
	GTM Lead Intelligence — NextAuth Integration, Multi-User Isolation & Custom Signin Page (29 Sep) Implemented the full NextAuth Azure AD authentication in code. Configured NextAuth options with Azure AD provider (clientId, clientSecret, tenantId), JWT session strategy, and three callbacks — signIn (upserts user to DB and creates default preferences on first login), jwt (attaches userId to token), and session (exposes userId to client). Built user repository with upsert-by-email logic. Implemented automatic preference creation on first login — new users get default preferences (APAC region, All industries, Concierto Migrate offering) without manual setup. Enforced multi-user data isolation across all API endpoints — every query for leads, preferences, signals, and search history now filtered by authenticated user's ID. Built the custom /auth/signin page with Trianz branding, Microsoft login button, and time-based greeting. Wrapped the app in SessionProvider with redirect to signin for unauthenticated users.	**Done**	Full SSO flow working; isolation enforced across endpoints
	GTM Lead Intelligence — 5-Product Optimization Planning & GPT-4o Prompt Architecture (30 Sep) Conducted a comprehensive review of the existing lead generation setup — identified that only 3 offerings were partially configured with no signals for Manage or Maximize. Planned the full 5-product optimization covering all layers: DB signals, UI dropdowns, query generation, and GPT-4o prompt. Designed the 45-signal matrix across all 5 offerings (7 Migrate, 7 Modernize, 7 Manage, 6 Maximize, 9 Intelligence) with priority ranking (10–30 scale). Architected the new GPT-4o system prompt — 5-product definitions, customer pain-to-product mapping table, 6-step evaluation framework (Identify Buyer → Detect Problem State → Map Offering → Score Lead → Disqualify → Validate), and disqualification rules. Designed the Problem State Detection model: State A (facing problem, high urgency), State B (actively solving, active opportunity), State C (already solved, evaluate next-phase product). Planned geo-keyword query enhancement and region-aware search query generation.	**Done**	Full optimization plan fi 5 offerings, 45 signals, p architecture ready for implementation
	GTM Lead Intelligence — Full 5-Product Optimization Execution & Azure Deployment (1 Oct) Executed the complete 5-product optimization and deployed all changes to production. Implemented Azure AD SSO with full multi-user support — user upsert on first login, automatic preference creation, and data isolation across all API endpoints. Fixed a Next.js 16 middleware conflict blocking the auth flow on Azure. Rewrote the GPT-4o system prompt with all 5 Concierto products, pain-to-product mapping, 6-step	**Done**	All 5 offerings live on pr fixes and optimizations d a single day

	evaluation framework, and Problem State A/B/C detection. Updated the full UI — offerings dropdown (5 products), industries dropdown (9 Trianz verticals). Rewrote query generation with geo-keywords (North America / APAC / EMEA) and 'enterprise' qualifier. Applied DB migration force-upserting all 45 signals across 5 offerings. Fixed a critical bug where discovery runs completed instantly with 0 articles when no signals were in DB. Fixed migrations not running on Azure by updating app startup to run migrateDatabase() automatically on every deployment. Implemented Problem State C next-phase redirect logic.		
Week 18 *5 Oct*	**GTM Lead Intelligence — End-to-End Verification All Offerings & Regions + Lead Quality Optimization (5 Oct)** Conducted full end-to-end verification of lead generation across all 5 offerings and all 3 regions. Concierto Migrate — 22+ leads generated (North America); strong leads: SAP NS2 ($1B Army contract), NGA/HII ($84M cloud migration), DISA (data center consolidation), Customs & Border Protection (cloud migration RFP). Concierto Modernize — 16+ leads (North America); strong leads: Campbell Soup (SAP HANA on Azure), Kyndryl (US govt modernization), Mondelēz (digital overhaul). Concierto Manage — 3 leads (North America) + APAC leads (MUFG Bank Singapore, CLP Power Hong Kong); strong leads: Lupin (HPE cloud cost optimization), US Air Force (Oracle Fusion Cloud). Concierto Maximize — 4 leads; strong leads: Boeing (AI strategy), CIBC (AI strategy for banking), Douglas Elliman (Google Cloud AI transformation). Concierto Intelligence — 50+ leads (North America); strong leads: Lockheed Martin (enterprise GenAI), Toyota (AI agents), GE Appliances (Google Gemini manufacturing), Accenture/DoD ($821M War Data Platform). APAC region confirmed working. Implemented Problem State C next-phase redirect logic — companies that have already solved a problem now redirect to the logical next Concierto product; State C leads capped at Warm score. Vendor/provider announcements explicitly disqualified. Lead quality improved from ~6.5/10 to ~8.5/10.	**Done**	All 5 offerings + 3 regio live; 203+ total leads in s

Legend — Status: Done | In Progress | Blocked | Planned